

CSE 20 Discussion: Week 2

Data types, defining sets, defining functions

Overview: data types

- ▶ A **set** is an unordered collection of elements. Repetition doesn't matter; equal sets agree on membership of all elements.
- ▶ An n -**tuple** is an ordered sequence of elements with n slots ($n > 0$). Repetition matters and the length is fixed; equal n -tuples have corresponding components equal. A 2-tuple is an **ordered pair**.
- ▶ A **string** is an ordered finite sequence of elements, each from a specified set called the alphabet. Repetition matters and the length is an arbitrary finite number; equal strings have the same length and corresponding characters equal.
- ▶ Special objects: the **empty string** λ (a string of length 0) and the **empty set** $\{\}$ or $\emptyset$ (a set with no elements).
- ▶ Data types determine meaning: xy is string concatenation when x, y are strings and multiplication when x, y are numbers.
- ▶ Modeling involves choosing data types to represent and organize data.

Overview: defining sets

- ▶ Sets of numbers: $\mathbb{R}$, $\mathbb{Z}$, $\mathbb{Z}^+$ (strictly positive integers), $\mathbb{N}$ (nonnegative integers; $0 \in \mathbb{N}$).
- ▶ **Roster method**: explicitly list the elements, e.g. $\{A, C, U, G\}$.
- ▶ **Set builder**, two forms:
 - ▶ $\{x \in U \mid \dots x \dots\}$: all x from the universe U such that $\dots$
 - ▶ $\{\dots x \dots \mid x \in U\}$: all outputs of an operation as the input ranges over U .
- ▶ **Recursive definition**: a basis step (finitely many elements) and a recursive step (rules for new elements from existing ones). The set consists of *all and only* the elements obtained by finitely many applications of the recursive step after the basis step.
- ▶ **Set operations**: the Cartesian product $X \times Y$ (elements are ordered pairs) and set-wise concatenation $X \circ Y$ (elements are strings).

Overview: defining functions

- ▶ A function is defined by its **domain**, its **codomain**, and a **rule** assigning each element of the domain *exactly one* element of the codomain. Notation: $\text{FUNCTION-NAME} : \text{DOMAIN} \rightarrow \text{CODOMAIN}$.
- ▶ The rule can be a table, a formula, a piecewise definition, or an English description. Not every element of the codomain needs to be an output.
- ▶ When the domain is a recursively defined set, the rule can be defined recursively:
 - ▶ $\text{rnanlen} : S \rightarrow \mathbb{Z}^+$ computes the length of an RNA strand.
 - ▶ $\text{basecount} : S \times B \rightarrow \mathbb{N}$ counts the occurrences of a base. Its input is a *single* ordered pair (s, b) .

Part 1: Data types and defining sets

Q1. Same or different?

Decide whether each statement is true or false. Justify your answer using a property of the relevant data type(s).

(a) $\{1, 2, 2\} = \{2, 1\}$

(b) $(A, C) = AC$

(c) $\{\} = \{\lambda\}$

(d) The set $\{(0, 0), (0, 0, 0), 0, (0, 0)\}$ has exactly three distinct elements.

Q2. Translating between English and set notation

Consider the set described in English as

“the set of all ordered pairs of natural numbers whose second component is twice their first component.”

(a) Write this set in set builder notation of the form $\{x \in U \mid \dots x \dots\}$.

(b) Write the same set in set builder notation of the form $\{\dots x \dots \mid x \in U\}$.

(c) Give one element of this set and one ordered pair of natural numbers that is not an element of it.

Q2. Translating between English and set notation (continued)

(d) Describe $\{(x, x, x) \mid x \in \{-1, 0, 1\}\}$ in English and write it using the roster method.

(e) Describe $\{x \in \mathbb{Z}^+ \mid x \leq 0\}$ in English and write it using the roster method.

(f) Is $\{x \in \mathbb{Z} \mid x \geq 0\}$ equal to $\mathbb{N}$? Is it equal to $\mathbb{Z}^+$? Justify each answer.

Q3. Which set does a recursive definition define?

The set T is defined (recursively) by:

Basis Step: $0 \in T$

Recursive Step: If $x \in T$, then $x + 3 \in T$ and $x - 5 \in T$

(a) Show that $1 \in T$ and $-1 \in T$ by giving, for each, a finite sequence of applications of the recursive step after the basis step.

(b) Explain why every integer is an element of T . Conclude that $T = \mathbb{Z}$.

Q4. Recursive definitions of sets of strings

Let S be the set of RNA strands and $B = \{A, C, U, G\}$, as defined in class.

(a) Is $\lambda \in S$? Is $\lambda \in \{0, 1\}^*$? Justify each answer using the recursive definitions.

(b) Give a recursive definition of the set T_A of all RNA strands whose first base is A.

Q4. Recursive definitions of sets of strings (continued)

(c) A student proposes the following definition for T_A :

Basis Step: $A \in T_A$

Recursive Step: If $s \in T_A$, then $As \in T_A$

Does this define the set of all RNA strands whose first base is A? If not, give a strand that is in one set but not the other.

Q5. Modeling with data types

Let $C = \{(r, g, b) \mid 0 \leq r \leq 255, 0 \leq g \leq 255, 0 \leq b \leq 255, r \in \mathbb{N}, g \in \mathbb{N}, b \in \mathbb{N}\}$ be the set of colors, where r , g , b are the red, green, and blue components.

Suppose a color were instead modeled as the set $\{r, g, b\}$. Give two different colors in C that this model would represent by the same set, and explain which property of sets causes the problem.

Part 2: Defining functions

Q6. Defining functions on RNA strands

Let $gc : S \rightarrow \mathbb{N}$ be the function that gives the total number of Gs and Cs appearing in an RNA strand.

(a) Give a rule for gc that uses *basecount* (and does not use recursion directly).

(b) Give a recursive definition of gc , in the style of the definition of *rnalen*.

(c) Why is the basis step of *basecount* stated for inputs (b_1, b_2) with $b_1 \in B$, rather than for (λ, b_2) ?

Appendix: Definitions from the lecture notes

Definitions for Q1

set: unordered collection of elements; *repetition doesn't matter; Equal sets agree on membership of all elements.*

n -tuple: ordered sequence of elements with n “slots” ($n > 0$); *repetition matters, fixed length; Equal n -tuples have corresponding components equal.*

string: ordered finite sequence of elements each from specified set (called the alphabet over which the string is defined); *repetition matters, arbitrary finite length; Equal strings have same length and corresponding characters equal.*

Special cases: When $n = 2$, the 2-tuple is called an **ordered pair**. A string of length 0 is called the **empty string** and is denoted λ . A set with no elements is called the **empty set** and is denoted $\{\}$ or $\emptyset$.

◀ Back to Q1

Definitions for Q2

Sets of numbers:

- $\mathbb{R}$ The (set of all) real numbers (numbers on the number line)
- $\mathbb{Z}$ The (set of all) integers (whole numbers including negatives, zero, and positives)
- $\mathbb{Z}^+$ The (set of all) strictly positive integers
- $\mathbb{N}$ The (set of all) natural numbers (nonnegative integers). *We use the convention that 0 is a natural number.*

Roster method: To define a set using **roster method**, explicitly list its elements. That is, start with $\{$ then list elements of the set separated by commas and close with $\}$.

Membership: We use the symbol $\in$ as “is an element of” to indicate membership in a set.

[◀ Back to Q2](#)

Definitions for Q2

Set builder definition: To define a set using **set builder definition**, either form “The set of all x from the universe U such that x is ...” by writing

$$\{x \in U \mid ...x...\}$$

or form “the collection of all outputs of some operation when the input ranges over the universe U ” by writing

$$\{...x... \mid x \in U\}$$

[◀ Back to Q2](#)

Definitions for Q2

Cartesian product: Let X and Y be sets. The **Cartesian product** of X and Y , denoted $X \times Y$, is the set of all ordered pairs (x, y) where $x \in X$ and $y \in Y$

$$X \times Y = \{(x, y) \mid x \in X \text{ and } y \in Y\}$$

Conventions: (1) Cartesian products can be chained together to result in sets of n -tuples and (2) When we form the Cartesian product of a set with itself $X \times X$ we can denote that set as X^2 , or X^n for the Cartesian product of a set with itself n times for a positive integer n .

Pro-tip: the meaning of writing one element next to another like xy depends on the data-types of x and y . When x and y are strings, the convention is that xy is the result of string concatenation. When x and y are numbers, the convention is that xy is the result of multiplication.

◀ Back to Q2

Definitions for Q3

Recursive Definitions of Sets: The set S (pick a name) is defined by:

Basis Step: Specify finitely many elements of S

Recursive Step: Give rule(s) for creating a new element of S from known values existing in S , and potentially other values.

The set S then consists of all and only elements that are put in S by finitely many (a nonnegative integer number) of applications of the recursive step after the basis step.

Definition: The set of nonnegative integers $\mathbb{N}$ is defined (recursively) by:

Basis Step: $0 \in \mathbb{N}$

Recursive Step: If $n \in \mathbb{N}$, then $n + 1 \in \mathbb{N}$

Definition: The set of all integers $\mathbb{Z}$ is defined (recursively) by:

Basis Step: $0 \in \mathbb{Z}$

Recursive Step: If $x \in \mathbb{Z}$, then $x + 1 \in \mathbb{Z}$ and $x - 1 \in \mathbb{Z}$

Definitions for Q4

Recursive Definitions of Sets: The set S (pick a name) is defined by:

Basis Step: Specify finitely many elements of S

Recursive Step: Give rule(s) for creating a new element of S from known values existing in S , and potentially other values.

The set S then consists of all and only elements that are put in S by finitely many (a nonnegative integer number) of applications of the recursive step after the basis step.

RNA: RNA is made up of strands of four different bases that encode genomic information in specific ways. The bases are elements of the set $B = \{A, C, U, G\}$. Strands are ordered nonempty finite sequences of bases.

Definition: The set of RNA strands S is defined (recursively) by:

Basis Step: $A \in S, C \in S, U \in S, G \in S$

Recursive Step: If $s \in S$ and $b \in B$, then $sb \in S$
where sb is string concatenation.

Definitions for Q4

Definition: The set of bitstrings (strings of 0s and 1s) is defined (recursively) by:

Basis Step: $\lambda \in X$

Recursive Step: If $s \in X$, then $s0 \in X$ and $s1 \in X$

Notation: We call the set of bitstrings $\{0, 1\}^*$ and we say this is the set of all strings over $\{0, 1\}$.

Special cases: When $n = 2$, the 2-tuple is called an **ordered pair**. A string of length 0 is called the **empty string** and is denoted λ . A set with no elements is called the **empty set** and is denoted $\{\}$ or $\emptyset$.

[◀ Back to Q4](#)

Definitions for Q5

set: unordered collection of elements; *repetition doesn't matter; Equal sets agree on membership of all elements.*

n -tuple: ordered sequence of elements with n “slots” ($n > 0$); *repetition matters, fixed length; Equal n -tuples have corresponding components equal.*

Conclusion: Modeling involves choosing data types to represent and organize data

◀ Back to Q5

Definitions for Q6

Definition: The set of RNA strands S is defined (recursively) by:

Basis Step: $A \in S, C \in S, U \in S, G \in S$

Recursive Step: If $s \in S$ and $b \in B$, then $sb \in S$
where sb is string concatenation.

Definition: (Of a function, recursively) A function $rnalen$ that computes the length of RNA strands in S is defined by:

	$rnalen : S \rightarrow \mathbb{Z}^+$
Basis Step: If $b \in B$ then	$rnalen(b) = 1$
Recursive Step: If $s \in S$ and $b \in B$, then	$rnalen(sb) = 1 + rnalen(s)$

Piecewise rule definition: $f(x) = \begin{cases} \text{rule 1 for } x & \text{when COND 1} \\ \text{rule 2 for } x & \text{when COND 2} \end{cases}$: Define f of x to be the result of applying rule 1 to x when condition COND 1 is true and the result of applying rule 2 to x when condition COND 2 is true. This can be generalized to having more than two conditions (or cases).

Definitions for Q6

Example: A function *basecount* that computes the number of a given base b appearing in a RNA strand s is defined recursively:

$$\begin{array}{lll} & \text{basecount} : S \times B & \rightarrow \mathbb{N} \\ \text{Basis Step:} & \text{If } b_1 \in B, b_2 \in B & \text{basecount}((b_1, b_2)) = \begin{cases} 1 & \text{when } b_1 = b_2 \\ 0 & \text{when } b_1 \neq b_2 \end{cases} \\ \text{Recursive Step:} & \text{If } s \in S, b_1 \in B, b_2 \in B & \text{basecount}((sb_1, b_2)) = \begin{cases} 1 + \text{basecount}((s, b_2)) & \text{when } b_1 = b_2 \\ \text{basecount}((s, b_2)) & \text{when } b_1 \neq b_2 \end{cases} \end{array}$$

◀ Back to Q6